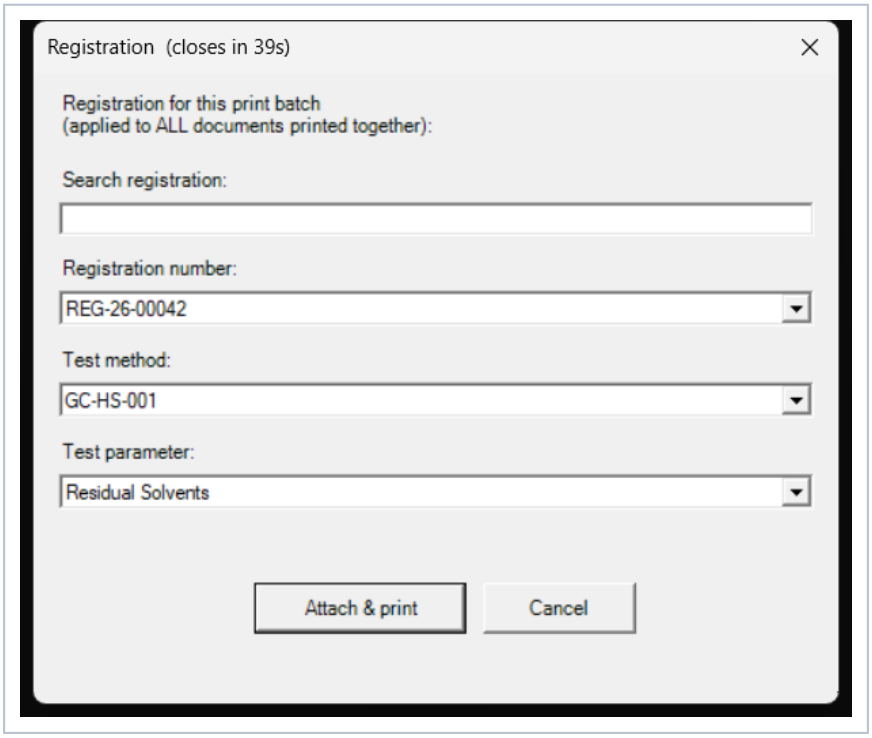
LIMS Virtual-Printer Pipeline

End-to-End Guide & Operations Manual

Print → Hub → Supabase LIMS • the printer_data model



The per-print registration dialog every user sees (batch mode shown).

Covers: architecture, deployment, every feature, every user & admin flow, and a full catalogue of edge cases and how the system handles each.

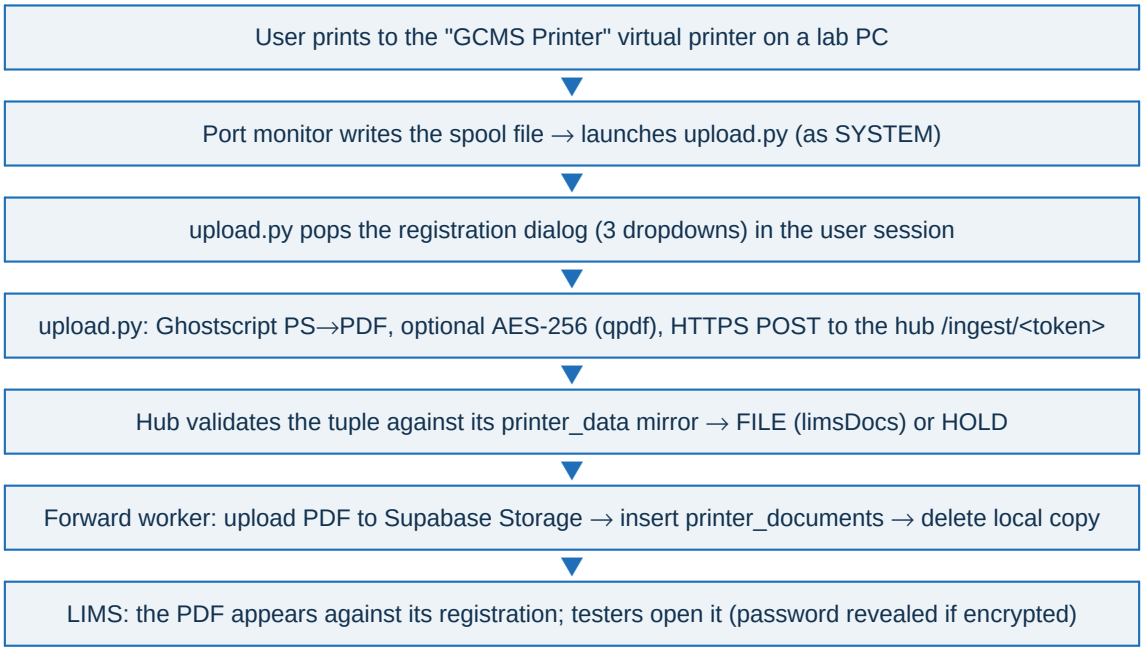
Generated as a working reference for the reworked (printer_data / printer_documents) pipeline. The three UI images in this guide are live screenshots captured from the actual dialogs; the hub dashboard is documented control-by-control (its browser view could not be image-captured in the build environment).

1 What this system is

A lab PC prints to a **virtual printer**. Instead of paper, the job becomes a PDF, is tagged with a **registration number, test method and test parameter**, and is delivered to the central **LIMS Print Hub** on the LAN. The hub files it, then pushes it to the cloud **Supabase LIMS** (Storage + a `printer_documents` row) and deletes its local copy. Testers find the PDF in the LIMS against the right registration.

The whole pipeline is driven by one flat feed the LIMS owns, **printer_data**, keyed on **department + equipment + registration + method + parameter**. A printer is enrolled to a **department + equipment** (e.g. Chemistry / GCMS) and only ever sees the registrations valid for it.

1.1 The pipeline at a glance



1.2 The three tiers

Client (each lab PC)	The virtual printer, the port monitor, `upload.py` (stdlib-only, runs as SYSTEM), and `prompt-id.ps1` (the dialog). Installed by the GUI wizard. Windows 7 SP1 / 10 / 11.
Hub (one central desktop)	`hub/app.py` - a FastAPI service. Mirrors `printer_data`, receives jobs, files them into the `limsDocs` tree, and forwards to Supabase. Has a web dashboard.
Cloud LIMS (Supabase)	The megascan LIMS project. Owns `printer_data` (built from registrations), stores pushed PDFs in the `printer-documents` bucket + `printer_documents` table, and per-device PDF passwords (encrypted) in `printer_devices`.

2 The data model (Supabase)

These objects live in the LIMS Supabase project (migrations on branch `f/print`). The hub reads them over PostgREST with the service-role key.

printer_data	The flat feed the hub reads. Columns (all NOT NULL): `registration_number, department_name, equipment_name, status, test_method, test_parameter`. Primary key = (registration_number, department_name, test_method, test_parameter).
test_parameter_equipment	The manual test→instrument bridge: `test_parameter` (PK) → `equipment_name`. The LIMS has no direct test→equipment relation, so this supplies `printer_data.equipment_name`.
printer_devices	Per-printer AES-256 PDF password, **encrypted at rest** (pgcrypto, key in Supabase Vault). Set/reveal only via the admin-gated RPCs `set_device_password` / `reveal_device_password`.
printer_documents	One row per pushed PDF: `pdf_name, pdf_from, device_name, equipment_name, registration_number, test_method, department_name, test_parameter, printed_by, size_of_pdf, received_time, storage_path` (unique - the idempotency key).
refresh_printer_data() + triggers	Rebuilds `printer_data` from `sample_registrations` (reg_no, method via test_method_id, parameters + department via dept_param_map, equipment via the bridge). Fires on any change to the source tables.
printer_data_version()	A cheap text watermark (md5 of all rows + count). The hub polls it (~2s) and refetches only when it changes.
Storage bucket printer-documents	Private. The hub uploads PDFs here (object key = the limsDocs dir path); the LIMS downloads via signed URLs.

NOTE How printer_data fills: a registration must be routed at Masters Review (its `dept_param_map` set) AND every printable test_parameter must have a row in `test_parameter_equipment`. A parameter with no equipment mapping is deliberately skipped (its instrument is unknown).

3 Deployment (one-time setup)

3.1 Part 1 - apply the Supabase migrations

On the megascan Supabase project, apply the migrations from branch `f/print` (either `supabase db push` or paste them into the SQL editor, in filename order):

- `printer_data`, `test_parameter_equipment` (the feed + bridge)
- `printer_devices` + `set/reveal_device_password` RPCs (needs the Vault + pgcrypto extensions, preinstalled on Supabase)
- `printer_documents` + the `printer-documents` Storage bucket
- `printer_data_version()` and `refresh_printer_data()` + triggers

> **IMPORTANT** The migrations assume the base megascan schema already exists (the `sample_registrations`, `test_methods`, `test_parameters`, `departments` tables, the `app_resource` enum with an `operations` value, and helpers `has_permission()` / `touch_updated_at()`). Apply them on top of that schema.

3.2 Part 2 - populate printer_data

Populated automatically by `refresh_printer_data()`. To make a registration appear in printers:

- Route the registration at Masters Review so its `dept_param_map` (which department runs which parameter) is set.
- For every test parameter that will be printed, add a row to `test_parameter_equipment` mapping it to the instrument (e.g. `('Residual Solvents','GCMS')`).

The triggers re-project `printer_data` on any change; the hub picks it up within ~2s.

3.3 Part 3 - run the hub (central desktop, 192.168.1.172)

```
cd hub
run.bat        (prints the ADMIN TOKEN + ENROLL KEY on first start)
```

- Open the dashboard at `http://<hub-ip>:8000/` and paste the admin token (top-right).
- Settings tab: set the **limsDocs directory** (local path of the shared folder), the **Supabase URL**, and the **service-role key** (stored DPAPI-encrypted). Save.
- Within ~2s the Catalog tab shows the `printer_data` rows synced from Supabase.
- Share the limsDocs folder so clients can read `\\<hub>\limsDocs\.vcp\catalog`, and allow inbound TCP 8000 in the firewall.

> **NOTE** No Supabase configured = **local mode**: the hub runs, but `printer_data` stays empty until a URL+key are set. The forward queue idles and drains once configured. Nothing breaks.

3.4 Part 4 - install a printer on each lab PC

Double-click `install_10_11.bat` (Windows 10/11) or `install_7.bat` (Windows 7 SP1). Each self-elevates and opens the wizard. See section 5.

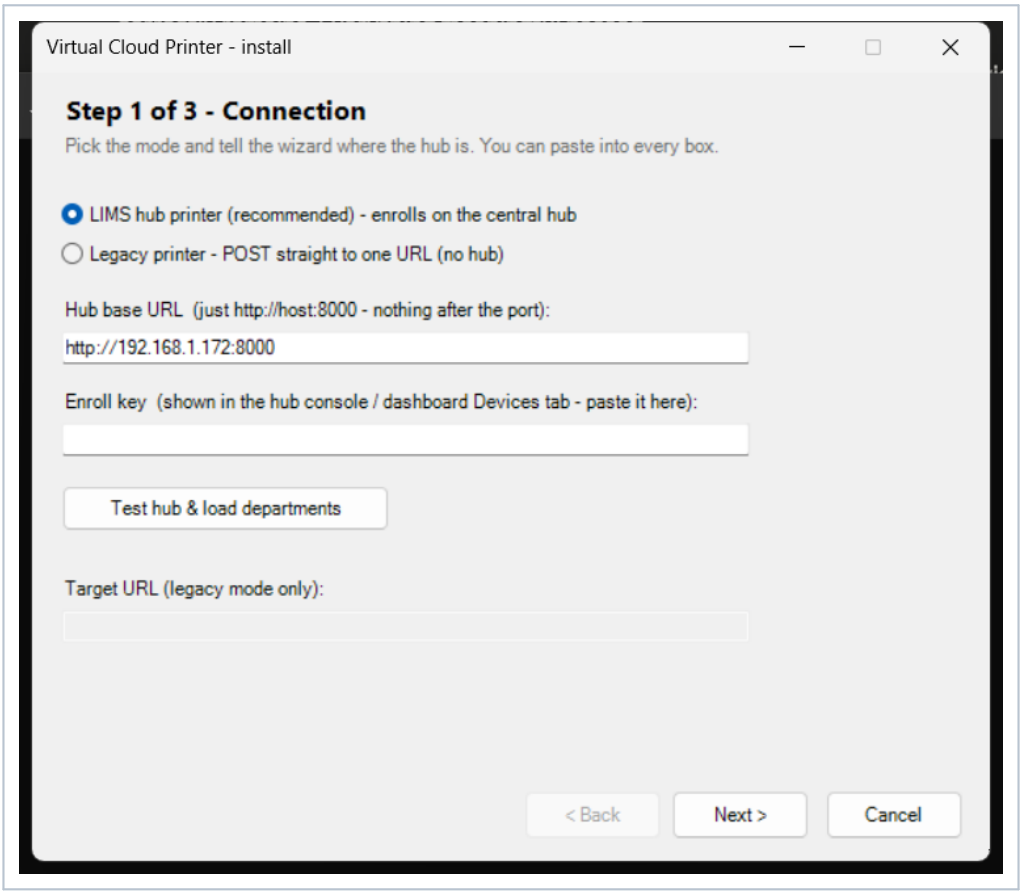
4 The hub dashboard

Served at `http://<hub>:8000/`. Paste the admin token once (top-right, "Save"); it is remembered for the session and auto-refreshes every 3s. Five tabs:

Devices	Every enrolled printer: **Name, Department, Equipment, Printer, Host, Ingest URL, Docs, Last seen**. "Show enroll key" reveals the key for installs; "revoke" removes a device's token (its documents remain).
Documents	Every job: **Document** (the stored file name + "from: <print title>"), **Device** (name + department/equipment), **Reg / Method / Parameter**, **Status** (filed / held / forwarded / forward_failed), **Printed by**, **Size**, **Received (IST)**, and a download link. Filter by status.
Held	Jobs the hub could not validate. Each row has cascading **Registration** → **Method** → **Parameter** dropdowns (filtered to the doc's department+equipment); pick all three and click **Assign** to file & forward it. Download to inspect first.
Catalog	A read-only view of the `printer_data` mirror with a filter box: Registration, Department, Equipment, Status, Method, Parameter. This is what drives every printer's dropdowns.
Settings	limsDocs directory, Supabase URL + service-role key (write-only; blank keeps the current key), and catalog poll seconds.

> **NOTE** The Catalog tab is **read-only** - `printer_data` is owned by the LIMS. To change what printers can select, edit the registration/routing in the LIMS (or the `test_parameter_equipment` bridge); the hub re-syncs automatically.

5 Installing a printer (the wizard)



Step 1 of 3 - Connection. Hub mode is the default; the hub URL is pre-filled.

5.1 Step 1 - Connection

- Choose **LIMS hub printer** (default) or Legacy (a plain POST-to-one-URL printer, unchanged from the old tool).
- Enter the **Hub base URL** (just `http://host:8000`) and paste the **Enroll key** (from the hub console / Devices tab).
- Click **Test hub & load departments** - it validates the key and loads the department list.

5.2 Step 2 - Printer & device

- **Printer name** - how it appears in the Windows print dialog (e.g. "GCMS Printer").
- **Device name** - unique on the hub (e.g. `gcms-01`); validated live.
- **Department** - pick from the loaded list; choosing one loads its **Equipment** list.
- **Equipment** - the instrument (GCMS, LCMS, ...). Department + Equipment decide which registrations this printer sees.
- **Set PDF password** (optional) - if set, every PDF from this printer is AES-256 encrypted, and the password is stored (encrypted) in Supabase for testers to reveal. Leave blank for no encryption. Confirm it in the repeat box.

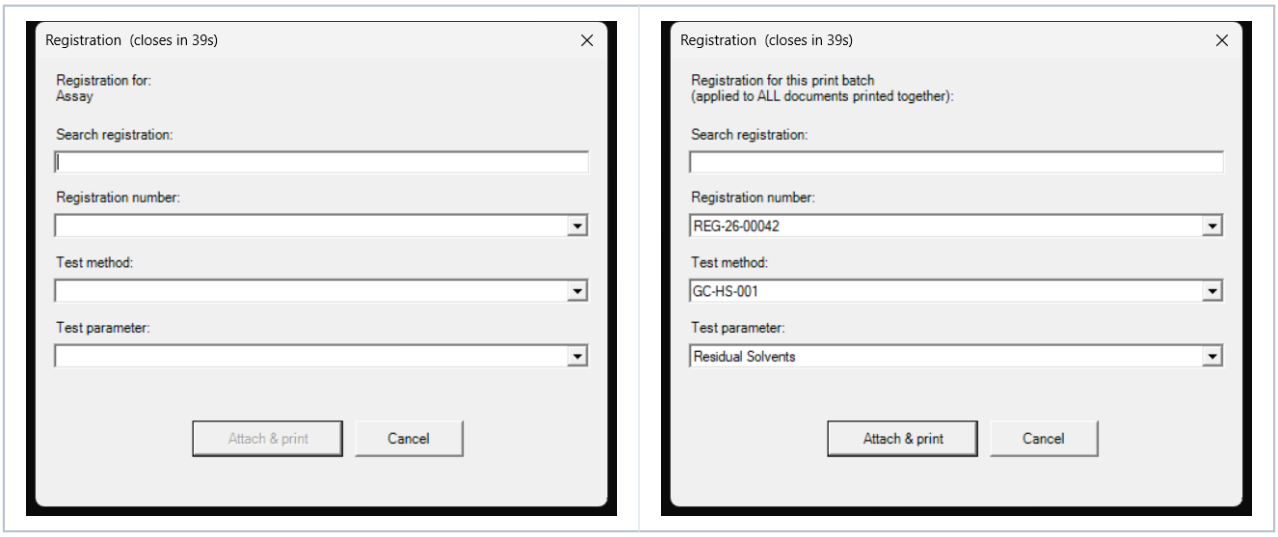
5.3 Step 3 - Review & install

Confirm the summary and click **Install**. First run downloads tools (uv, Ghostscript, the port monitor, qpdf) and can take a few minutes; progress streams in the window. On success the printer is ready to print.

> **NOTE** Enrollment stores the device (department + equipment) on the hub and, if a password was set, calls `set_device_password` so the LIMS can reveal it. If Supabase is briefly unreachable the install still succeeds - the client keeps the password locally to encrypt; re-run to store it in the LIMS later.

6 Printing - the registration dialog

Print to the virtual printer from any app. A dialog pops up on the user's screen for the job:



Left: empty - Attach & print stays disabled until all three are chosen. Right: filled/ready (batch mode).

6.1 How the three dropdowns work

- **Registration number** - the open registrations for this printer's department+equipment. Type in the **Search** box to filter a long list.
- **Test method** - the methods available for the chosen registration (auto-selected if there is only one).
- **Test parameter** - the parameters for the chosen method (auto-selected if only one).
- All three are **required**; **Attach & print** only enables when every dropdown has a value.

6.2 Batch printing

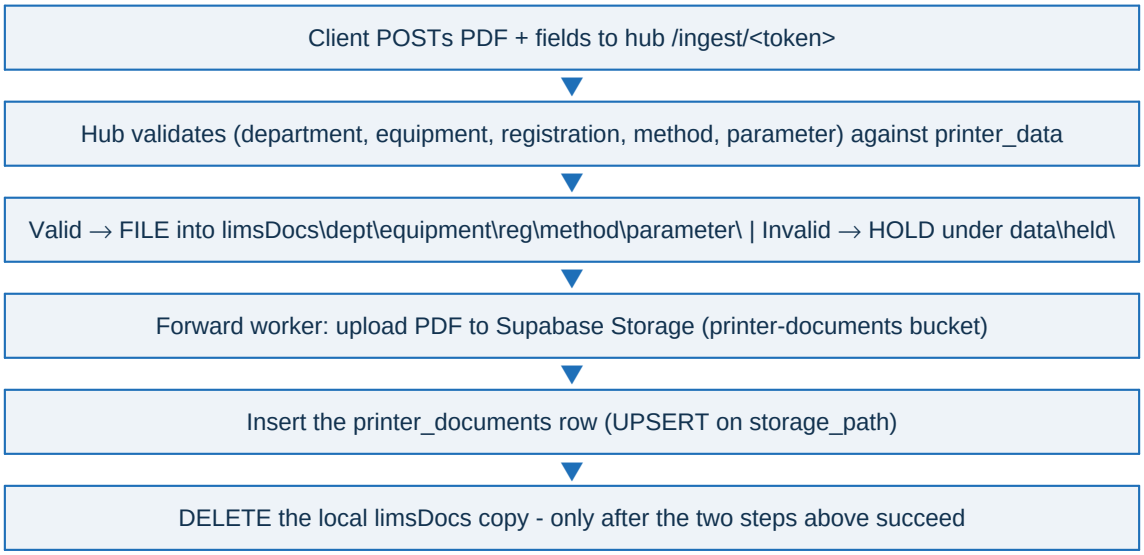
If one app prints several PDFs at once, **one dialog** appears (batch mode - the header says so) and its answer applies to the whole batch. The next document you print re-prompts (it never silently reuses the previous number).

6.3 Cancel vs. close vs. timeout

Cancel / close the window	The job is **DISCARDED** - nothing is converted, uploaded, or kept. A deliberate "I did not mean to print".
Timeout (nobody at the PC)	After ~3 min the dialog auto-closes writing nothing; the job proceeds without a number and the hub **holds** it (an unattended print is never destroyed).
Catalog unreachable	The dialog falls back to the share copy (`\\hub\limsDocs\.vcp\catalog\<dept>__<equipment>.json`); if that is missing too, it degrades to free-text entry and the hub validates/holds.

7 Filing, pushing & deleting (no data loss)

After the dialog, the client converts and uploads to the hub. The hub validates and either files or holds:



7.1 Why nothing is ever lost (concurrency-safe)

- **Idempotency:** the object key (`storage_path`) is fixed once and is UNIQUE. A retried upload returns 409 (already there) = success; the row insert is an UPSERT. So a retry can never duplicate or lose a document.
- **Ordering:** the hub commits `status='forwarded'` and drops the queue entry **before** deleting the local file. A crash after the commit leaves only an orphan local file (harmless); a crash before it leaves the job `filed` with the file intact, so the next run re-pushes and finishes.
- **Retries:** any failure keeps the row in the hub's `forward_queue` and retries with exponential backoff (5s → 5min), forever. Held documents stay on disk until assigned.

GOOD A print job is safe at every step: discarded only by an explicit user Cancel; otherwise it is filed or held, and once delivered the local copy is removed - guaranteeing the LIMS is the single source of truth without ever dropping a document.

8 Held documents & assigning them

A job is **held** when the hub cannot match its (department, equipment, registration, method, parameter) tuple to an available `printer_data` row - e.g. the registration is not routed yet, the parameter has no equipment mapping, or the user typed a value in the free-text fallback. The PDF is kept under `data\held\` (never in the share) and the client still gets a success response.

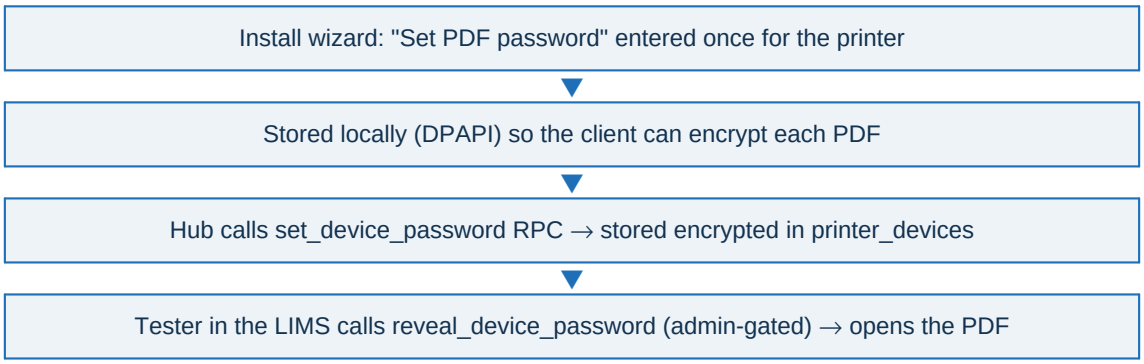
8.1 Assigning (Held tab)

- Open the **Held** tab; download the PDF to see what it is.
- Use the cascading **Registration** → **Method** → **Parameter** dropdowns (filtered to that document's department+equipment and to available rows only).
- Click **Assign**: the hub validates the tuple, moves the file into the limsDocs tree, and enqueues the push. The row moves to `filed` then `forwarded`.

missing_registration	No registration number was provided.
missing_method	No test method was provided.
missing_test	No test parameter was provided.
unknown_registration	The full tuple is not an available row in printer_data (unrouted, closed, or no equipment mapping).

9 Per-device PDF password (encryption)

If a printer is installed with a PDF password, every PDF it produces is AES-256 encrypted (via qpdf, lossless) before upload. The password is what opens the PDF, so it must be **recoverable** - it is stored **encrypted at rest** (pgcrypto, key held in Supabase Vault), never hashed, never in plaintext.



- **Change a password:** re-run the installer for that printer with a new password, or call `set_device_password` from the LIMS.
- **Reveal:** only via the admin-gated `reveal_device_password` RPC - the key never leaves the database or reaches the browser.
- **No password:** leave it blank at install; PDFs are uploaded unencrypted and there is no `printer_devices` row (that is fine).

> **CRITICAL** A SHA-256 hash was deliberately **not** used: a hash is one-way and could never open the encrypted PDF. Reversible encryption-at-rest is required and is what is implemented.

10 Edge cases & how each is handled

Situation	Handling
User clicks Cancel / closes the dialog	Job discarded entirely - not converted, uploaded, or kept. Deliberate.
Nobody at the PC (dialog times out)	After ~3 min proceeds with no number; hub HOLDS it. Never destroyed.
Hub catalog fetch fails at print time	Dialog loads the share fallback file `<dept>__<equipment>.json`; if missing, degrades to free-text; hub validates/holds.
Registration not yet routed (no dept_param_map)	It simply does not appear in printer_data until routed - printers can't select it.
Test parameter with no equipment mapping	Skipped from printer_data (instrument unknown). Such a job, if forced via free-text, is held; add a test_parameter_equipment row to fix.
Method whose only parameter is empty	Dropped from the catalog so the dialog's OK can always enable (no dead-end method).
Closed / completed registration	Hidden from printer dropdowns and rejected by validation (status not 'open'-like).
Concurrent prints from one user	Batch coalescing: one dialog, one answer for the whole burst; keyed per (user, printer).
Concurrent prints across many PCs (~100)	Each job has its own unique storage_path; the hub streams uploads and uses WAL SQLite - no collisions, no lost jobs.
Device enrolled with no password, or while Supabase was down	No printer_devices row is required - printer_documents.device_name is a plain snapshot (no FK), so forwarding still works.
Hub crash mid-push	storage_path idempotency + commit-before-delete ordering: re-push is a 409 no-op; the doc is never duplicated or stranded.
Supabase unreachable for a while	Jobs stay filed on disk; the forward queue retries forever with backoff and drains when connectivity returns.
Duplicate device name at enrollment	Rejected with HTTP 409 (case-insensitive) - pick a unique name.
Unknown department/equipment at enrollment	Rejected with HTTP 400 - it must exist in printer_data.
Registration renamed/deleted in the LIMS	Triggers re-project printer_data; the hub re-syncs within ~2s. Already-pushed documents keep their snapshot values.
Malformed dept_param_map	The refresh is guarded - it warns and leaves printer_data unchanged rather than failing; fix the routing data and re-save.
Windows Protected Print mode (Win11)	The installer refuses to run - it would block the port monitor. Turn WPP off first.

11 Troubleshooting

Printer dropdowns are empty	printer_data has no rows for that department+equipment. Check the Catalog tab; ensure the registration is routed and its parameters have equipment mappings.
Everything lands in Held	The tuple isn't in printer_data. Confirm the printer's department+equipment match the registration's, and the parameter→equipment mapping exists. Assign from the Held tab meanwhile.
Documents stuck at forward_failed	The hub can't reach Supabase or the key is wrong. Check Settings (URL + service-role key) and the row's error text; it keeps retrying.
Nothing reaches the hub at all	Confirm the printer name matches its config entry, Ghostscript resolves, and check `%ProgramData%\VirtualCloudPrinter\log.txt` and `failed\`. Run `setup.ps1 -Action fixqueue`.
Install hangs / GUI closes	Must run elevated (the .bat self-elevates). On Win11 disable Protected Print first. On Win7 install .NET 4.8 then WMF 5.1 first.
Tester can't open an encrypted PDF	Reveal the password from the LIMS (reveal_device_password). Confirm the printer was installed with a password and it was stored (re-run install if Supabase was down).

Primary debugging tools: the hub dashboard (Documents / Held), the hub console log, `setup.ps1 -Action status`, and `%ProgramData%\VirtualCloudPrinter\log.txt` on the client.

12 Quick reference

Hub ingest fields	registration_number, test_method, test_parameter, department_name, equipment_name, device_name, printed_by, pdf_from, job_id, sha256 (+ file)
Client config (per hub printer)	url (/ingest/<token>), token, device_name, department_name, equipment_name, pdf_encryption{enabled,password_dpapi}
Catalog watermark	POST /rest/v1/rpc/printer_data_version (~2s poll)
Storage tree / key	department / equipment / registration / method / parameter / <reg>_<method>_<param>_<uuid>.pdf
Install	install_10_11.bat (Win10/11) or install_7.bat (Win7 SP1) → GUI wizard
Repair the client queue	setup.ps1 -Action fixqueue

End of guide.